Decision Models as Tool Selectors for Coding Agents with Many MCP Tools

Aziz Muminov Alexandra Ilina

November 2026

Abstract

A coding agent connected to many MCP servers must either load every tool definition into its context, about 180k tokens for the 613 tools of twelve popular servers, or find tools through search. Claude Code lists tool names and searches by keyword; Codex, Anthropic's tool search and Hermes Agent use BM25; GitHub Copilot uses tool groups; Goose offered embedding and LLM routers in 2025 but has removed them. We ask whether a *decision model* (DM), which returns a probability per option for up to 255 options without generating text, is a better tool selector, using Microsoft Decision 1 on OpenRouter's Decisions API. We built a test suite from the real tool definitions of 30 MCP servers with simulated back ends, 38 coding tasks graded by hidden tests and required writes, and two agent harnesses, opencode and Hermes Agent. Our main experiment swaps the ranker inside Hermes Agent's tool-search bridge over 1,076 tools: BM25, Microsoft Decision 1 with and without a "none" option, embeddings, a Claude Code-style ranker and random, in 684 pinned, paired runs. Every ranker, even random, passed 96–100% of runs, but ranking showed in tokens: random used 1.57× the tokens of BM25. The DM rankers and embeddings used 7–12% fewer tokens than BM25; only DM with the "none" option had a 95% CI excluding 1, 0.88× [0.78, 0.98], and not after Bonferroni correction for five comparisons. With the DM, agents needed 2.7 searches per needed tool against 3.4 with BM25. Offline, a server-then-tool DM hierarchy ranked best: at 613 tools it put the right tool first for 93% of queries, against 48–54% for name, BM25 and embedding search and 86% for an LLM router. A projection suggests the DM's per-search fee pays off for agent models costing more than about $0.03 per run.

1 Introduction

Through the Model Context Protocol (MCP) [18], coding agents read CI logs, error trackers, tickets, chat, dashboards, clusters and databases, and the LLM must see each tool's name, description and parameter schema before calling it. Twelve widely used servers expose 613 tools whose definitions take about 180k tokens per request.

Harnesses therefore use *tool search*: definitions stay out of the prompt and the agent finds tools on demand (§2). Claude Code defers MCP tools, lists only their names, and loads them through a ToolSearch tool [3]; Anthropic's API and OpenAI's Codex CLI offer BM25 search [2, 22]; GitHub Copilot groups tools into "virtual tools" that the agent activates [15]; Goose offered embedding-based and LLM-based tool routers as a preview in 2025 and has since removed them [6]; Hermes Agent defers MCP tools behind a BM25 search tool [20]. These ranking functions have not been compared on the same tools and tasks.

Decision models (DMs) suit this ranking: a DM reads a state and a question with named options and returns a probability per option in one forward pass, without generating tokens. We use Microsoft Decision 1, MSD1 for short, served as `microsoft/microsoft-decision-1` [16] through

OpenRouter's Decisions API [24]. But a DM accepts at most 255 options, fewer than a realistic catalog, and cannot write arguments.

We ask: *can a DM serve as a coding agent's tool selector, and is it better than the mechanisms harnesses use?* Our test suite (§4) uses the real `tools/list` output of 30 MCP servers, 613 tools from twelve servers and 1,076 with decoy servers such as GitLab beside GitHub, together with simulated back ends, 38 graded tasks, eight selection conditions and two harnesses. Our findings, with caveats in §6:

- **Main experiment (§5.1).** Six rankers in Hermes Agent's search bridge, two of them Microsoft Decision 1: DM, and DM + none, which may answer that no tool fits; 684 paired runs. Every ranker passed 96–100% of runs. Random ranking used 1.57× the tokens of BM25, so ranking matters. The DM rankers and embeddings leaned 7–12% below BM25; only DM + none's 95% CI excluded 1, at 0.88× [0.78, 0.98], its Bonferroni-corrected 99% CI [0.75, 1.02] did not, and DM + none and DM did not differ significantly. The saving comes from fewer LLM calls and smaller prompts; with the DM, agents needed fewer searches to reach each needed tool.
- **Offline (§5.3).** At 613 tools the DM hierarchy put the right tool first for 93% of 120 hand-written queries, against 48–54% for name, BM25 and embedding search and 86% for an LLM router, at a third of the router's cost; at 1,076 tools the figures were 88%, 45–49% and 82%. On agent-written queries the gap is smaller.
- **Live, eight mechanisms (§5.4).** With opencode every catalog condition passed 96–100% of runs, and search cut the largest prompt from about 182k tokens to 12–53k, so pass rate cannot separate rankers; tokens must be compared, paired.

Contributions. We contribute a test suite for tool selection in coding agents on real MCP tool definitions, with simulated back ends, graded tasks and an isolated sandbox; a Microsoft Decision 1 tool ranker that passes the 255-option limit by choosing the server first, integrated into a shipped harness's search bridge; and a comparison of current mechanisms and the DM, offline and in two agents, with a paired, pinned protocol for measuring token differences between agent configurations.

2 How harnesses select tools

Table 1 lists the mechanisms we compare. All but native loading give the agent a search tool and a way to call what it finds, usually with a short server overview, so it pays for definitions only on retrieval.

Hermes's bridge. Hermes Agent 0.19.0 defers MCP tools once their definitions exceed 10% of the context window, as 613 tools do. `tool_search` returns compact hits of a name and one line, `tool_describe` one full definition, and found tools are called directly with typed arguments. Our main experiment swaps only the ranker behind `tool_search` (Figure 1).

3 Decision models as tool selectors

The interface. A Decisions API request carries a *state*, here the agent's search query, and named questions; a `choice` question lists options, each with a free-text description, and returns a probability per option [24]. We describe each tool by its name, the first 300 characters of its description and its parameter names. The DM in all our experiments is Microsoft Decision 1, post-trained from

Table 1: Tool-selection mechanisms and the conditions that reproduce them. Used in: M, main experiment (§5.1); R, offline retrieval (§5.3); O, opencode (§5.4).

Condition	Shipped or proposed in	What the agent sees and does	Used in
none	control	no catalog tools at all	O
native	Cursor, Cline, opencode; Claude Code with search off	every tool definition in every request	O
cc	Claude Code ToolSearch [3]	names of all deferred tools; `select:a,b` by exact name, or keyword search with name matches weighted highest; our approximation, as the exact scoring is undocumented	M R O
bm25	Anthropic tool search [2], Codex CLI [22], Hermes [20]	Okapi BM25 [26] over names, descriptions and parameters	M R O
groups	GitHub Copilot virtual tools [15]	one `activate_<group>` tool per server, with servers over 40 tools split; activating loads the group's definitions	O
embed	RAG-MCP [11], Goose 2025, since removed [6]	cosine similarity of `text-embedding-3-small` vectors of query and tool	M R O
llm	Goose 2025, since removed [6]; AnyTool-style LLM routing [9]	DeepSeek V4.1 Flash reads one line per tool and names the best ones	R O
dm	this paper	Microsoft Decision 1 (`microsoft/microsoft-decision-1`) ranks servers, then tools; DM + none adds a "none" option (§3)	M R O
random	control	five random tools	M

Qwen3.5-9B [16]; it answered in about half a second in our runs and costs $0.042 per million input tokens.

The 255-option limit. The API rejects `choice` questions with more than 255 options, and MSD1 rejects requests beyond about 32k tokens. We compared four ways around the limit:

- **Hierarchy**, our default: ask which server has the needed tool, with each of the 12 or 30 servers described by its purpose; take the most likely servers, at most three, until they cover 90% of the probability mass; ask which of their tools fits; and score each tool by $P(\text{server}) \cdot P(\text{tool} \mid \text{server})$.
- **Tournament**: split the catalog into chunks of at most 250, keep the top five of each, and decide among the survivors.
- **BM25→DM** and **embed→DM**: shortlist 100 tools with BM25 or embeddings, then let the DM choose among them.

The "none" option. DM + none adds an option "none of these services" to the server question and returns no tools when $P(\text{none}) \geq 0.4$, otherwise at most five. The threshold was tuned on the offline benchmark (§5.3). Figure 1 shows the hierarchy inside Hermes's bridge.

Why the DM selects and does not call. A DM cannot write arguments, and most tool choices in a coding session are easy: predicting the next tool of a real session, MSD1 did worse than always choosing the shell, while always-loaded definitions took most of the first request (App. B). The DM belongs in the catalog: choosing which of hundreds of rarely used tools to show, while the LLM keeps the core tools and writes every call.

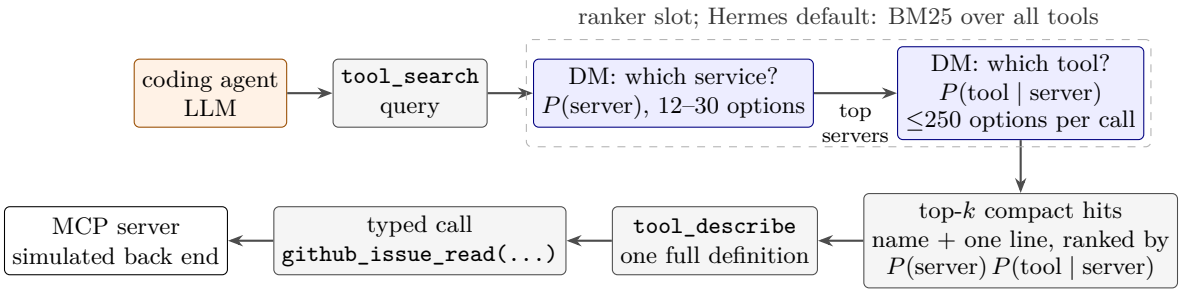


Figure 1: Tool selection with a decision model inside Hermes Agent's search bridge. The agent's query goes to the ranker slot, where the DM first chooses among servers and then among the tools of the most likely servers, with at most 250 options per call. The agent receives compact hits, reads one full definition with `tool_describe`, and calls the tool directly with typed arguments. In the default harness BM25 fills the same slot; everything outside the dashed box is identical across conditions.

4 Test suite

Real tool catalog. We started twelve MCP servers as an unprivileged user with dummy credentials and saved their `tools/list` responses: names, descriptions and input schemas exactly as published. No tool was ever executed against a real service. This main catalog has 613 tools from 12 servers, from Linear with 198 down to Postgres with one. Eighteen further servers add 463 tools, for 1,076 in all, including natural decoys such as GitLab beside GitHub, AWS CloudWatch beside Grafana Loki and AWS EKS beside Kubernetes; App. G lists all servers.

Simulated back ends. Each task has a world: rules that map a tool and matching argument values to a response, then organization-wide facts shared by all tasks; the first matching rule answers. Unmatched reads return an empty list or not found, as a real service would for something that does not exist; unmatched writes succeed and are recorded for grading. Optionally, arguments are validated against the tool's schema.

Tasks. Each task is a small Python repository whose fix needs information that lives only in the services, such as a failing CI check, a Sentry stack trace or a Slack decision, reachable only through the catalog since the sandbox has no network or credentials. Many tasks also require a write, such as commenting on the pull request or moving the ticket.

- **v1** has 33 tasks: 22 single-service, 3 multi-service, 4 *absent*, where the information does not exist and the right outcome is to stop and leave a `TODO`, and 4 controls that need no catalog tool. Prompts often name the service, as in "check the logs in Loki".
- **v2** has 38 tasks (App. G): the same worlds with *symptom-first* prompts that do not name the service, such as "customers stopped getting shipping e-mails"; a shared "where things live" handbook in every README, like an `AGENTS.md`, that names the company's tools including legacy GitLab, CloudWatch and EKS; and five new tasks, four multi-service and three on the extra servers, so that the decoy servers are sometimes the right answer.

The offline retrieval queries and the opencode runs use v1 with the 613-tool catalog; the main experiment uses v2 with all 1,076 tools. Every prompt ends with the same escape clause: if the information is not available, do not guess; leave the code unchanged and add a `TODO: info not found` comment. In the main experiment every prompt also starts, identically in all conditions, with a note standing in for what real harnesses say in the system prompt about connected MCP

Table 2: Experiments at a glance. Tasks: task set (§4); conditions as in Table 1.

Experiment	Harness	Tasks × tools	Agent LLM	Compared	Runs
Main (§5.1)	Hermes Agent 0.19.0	v2: 38 × 1,076	DeepSeek V4.1 Flash	6 rankers	684
Offline retrieval (§5.3)	none	120 queries × 613, 1,076	none; router: DeepSeek	8 rankers	–
opencode (§5.4)	opencode 1.19.4	v1: 33 × 613	DeepSeek V4.1 Flash, hy4	8 conditions	528

servers: "Note: the company's internal services (code hosting, CI, error tracking, metrics and logs, incidents and on-call, tickets, docs and runbooks, chat, clusters, databases) are connected to you as tools. Some tools are loaded on demand: if a tool you need is not listed, use tool search to find it."

Grading and validation. A run passes when the hidden tests pass, every required write was made with the right arguments, and no destructive call such as a merge, delete, rollout restart or `kubectl apply` was made. Before use, we checked that each task's hidden tests fail on the original repository and pass with the reference solution, that every tool it names exists, and that the reference calls get non-empty answers, or empty ones for absent tasks, and satisfy the required writes.

Harnesses. *opencode* [23] 1.19.4 runs headless with a shell limited to `pytest` and file tools limited to the project; our MCP server implements every condition of Table 1, returning the top five tools with full definitions and a service overview, with calls through `call_tool(name, arguments)`. *Hermes Agent* [20] 0.19.0 uses its own bridge (§2) over our server's full tool list; we patched its search function to ask our server's `/rank` endpoint, so only the ranker differs (Figure 1).

Isolation. Runs execute as an unprivileged user without credentials, reaching only the MCP server and the LLM proxy, which holds the API key; each Hermes run also has its own mount namespace, a per-run MCP token hidden from the agent's shell, and a deny list of probing commands (App. A).

5 Experiments

The main experiment asks whether the ranker changes accuracy or tokens in a shipped harness with everything else fixed; a projection reuses its token counts to ask at which agent-model price the DM's fee pays off. Offline retrieval removes the agent and asks which ranker puts the right tool first and how to take the DM past 255 options. The opencode runs ask whether the tasks need the catalog and how much context search saves against loading every tool (Table 2).

5.1 Main experiment: rankers inside Hermes's search bridge

This experiment tests whether the ranker behind tool search changes accuracy or tokens when everything else in a shipped harness is fixed. Hermes Agent's bridge (Figure 1) lets us swap the ranker alone.

Design. Everything is fixed except the ranker: Hermes Agent 0.19.0 with the isolation of §4; task set v2 with schema validation and the connected-services note; and one LLM, DeepSeek V4.1 Flash [8], pinned to DeepInfra's fp8 endpoint with temperature 0.6 and seed 7. The six rankers are

Table 3: Main experiment: Hermes Agent with pinned DeepSeek V4.1 Flash, task set v2, 1,076 tools, three repeats, 114 runs per ranker, re-graded. Tokens: all input and output tokens of a run, median per run. Ratio: geometric mean of per-slot token ratios against BM25 with a task-level bootstrap 95% CI. Fewer: slots out of 114 on which the ranker used fewer tokens than BM25. LLM calls and searches: mean per run; time: median wall time per run. Bold: best token values. Pass-rate differences amount to at most four runs and are not significant.

Ranker	pass@1	pass^3	Tokens	Ratio [95% CI]	Fewer	LLM calls	Time (s)	Searches
BM25	0.991	37/38	558k	–	–	29.0	187	11.4
DM + none	0.991	37/38	**459k**	**0.88** [0.78, 0.98]	**69**	27.4	196	10.1
DM	0.991	37/38	480k	0.93 [0.82, 1.04]	66	27.9	173	9.7
embeddings	0.965	35/38	562k	0.91 [0.81, 1.03]	63	27.9	175	11.1
Claude Code style	0.991	37/38	576k	1.02 [0.89, 1.16]	43	23.2	144	6.0
random	1.000	38/38	1,051k	1.57 [1.38, 1.79]	27	34.9	271	12.8

BM25, as Hermes ships it and Codex CLI also uses it; *DM*, Microsoft Decision 1 choosing server then tool and returning the top five; *DM + none*, the same with the "none" option of §3; *embeddings* with `text-embedding-3-small`, as in RAG-MCP; *Claude Code style*, which puts all deferred tool names in the prompt and offers keyword ranking and `select:<name>`; and *random*, five random tools as a lower bound. Each ranker ran every task three times, for $6 \times 38 \times 3 = 684$ runs; BM25, DM and random were queued repeat by repeat with each slot's conditions back to back, DM + none, Claude Code style and embeddings later in parallel batches. *Tokens* are all uncached input, cached input and output tokens summed over a run's LLM calls, as counted by Hermes; about 95% are cached input re-read on every turn, which is why token and cost reductions differ (§5.2). We pair runs by slot, a task and repeat number. Tables report medians per run; token ratios are geometric means of per-slot ratios with a task-level bootstrap 95% CI over 10,000 resamples; the projection uses means. We also report pass@1 and pass^3, which counts a task only if it passes in all three repeats.

Conditions not in the main experiment. Native loading is covered by the opencode runs (§5.4). We left out the LLM router and groups: no current harness ships an LLM router, the router cost about three times as much as the DM per query offline (Table 6), and groups load whole servers, giving 29–53k-token prompts in opencode against 12–35k for ranked search (Table 7).

Why paired, and why pinned. In pilot runs (App. D) the paired log ratio of tokens between two conditions had a standard deviation of about 0.44, so detecting a 10% token difference with 80% power at $\alpha = 0.05$ needs about 136 pairs; the design gives 114 per comparison. Unpinned, OpenRouter routed DeepSeek requests to dozens of endpoints with different quantizations, and the harness sets no temperature, hence both fixes. A second LLM, Kimi K3.1 Pro, was piloted and dropped for cost and speed.

Grading. We corrected two rules during the run and re-graded all runs from saved artefacts, which changed 33 of 684 verdicts: the destructive-call rule had matched any tool name containing "merge", blocking a required merge-request comment, and one hidden test in `cw_reports` had rejected a valid fix.

Accuracy is at ceiling for every ranker, including random. All six rankers passed 96.5–100% of runs and abstained correctly on all 12 absent-task runs (Table 3); given five useless tools, an

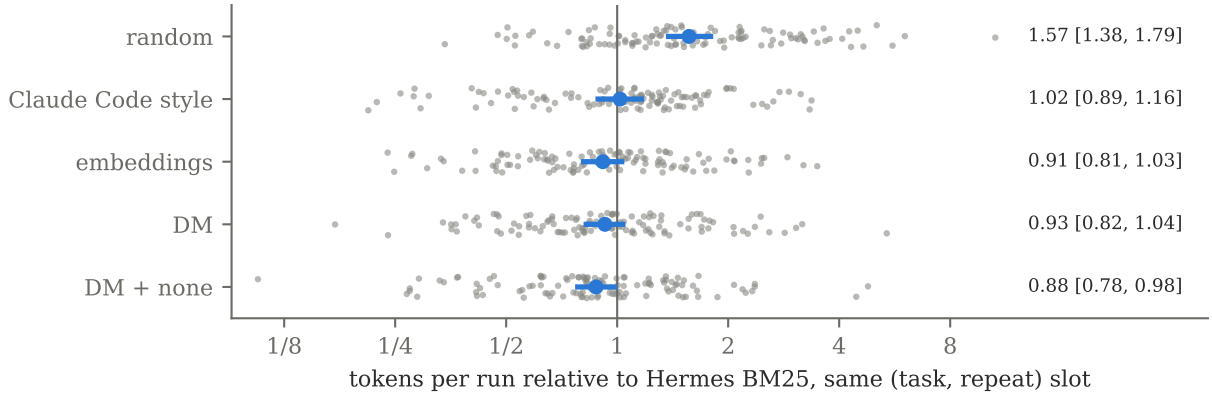


Figure 2: Tokens per run relative to BM25 in the same slot, 114 slots per ranker. Grey: single slots; blue: geometric mean with task-level bootstrap 95% CI, values at right. Single-slot ratios span roughly 0.25–4×, so only paired averages over many slots are informative: random is shifted clearly to the right, the learned rankers slightly to the left.

agent simply searches again. The remaining failures are real: a destructive `kubectl exec` in a production pod in `k8s_crashloop`, once each under BM25, DM and Claude Code style and twice under embeddings; a wrong fix in `cw_reports` under DM + none and embeddings; and unchanged code in `prom_threshold` under embeddings.

Ranking shows up in tokens. Random ranking cost 1.57× the tokens of BM25 [1.38, 1.79] and used fewer tokens on only 27 of 114 slots, so the ranker matters (Figure 2). The two Microsoft Decision 1 rankers and embeddings leaned 7–12% below BM25. DM + none was lowest, 0.88× [0.78, 0.98], with fewer tokens on 69 of 114 slots, and the only ranker whose 95% CI excludes 1; after Bonferroni correction for the five comparisons with BM25 it does not, with a 99% CI of [0.75, 1.02], and compared directly with DM, at 0.95× [0.84, 1.05], it is not significantly different. The intervals of DM and embeddings included 1, and embeddings also had the lowest pass rate. The Claude Code-style ranker tied with BM25 at 1.02×.

Where the saving comes from. DM + none runs made 0.94× the LLM calls of BM25 runs, 27.4 against 29.0, and sent smaller prompts per call, a median of 19.2k against 21.1k tokens, paired 0.93× [0.89, 0.97]. With the DM, agents reached each needed tool in fewer searches (Table 4): 2.7 on average against 3.4 with BM25, and 30% of needed tools took three or more searches against 46%. The paired difference is significant for DM, −0.71 searches per needed tool [−1.13, −0.25] over 90 slots, but not for DM + none, −0.25 [−0.83, 0.48], or embeddings, −0.13 [−0.68, 0.54]. With BM25 the agent more often had to rephrase and search again. Task categories have few tasks and give only a hint, but the "none" option helped where the information does not exist: on the 4 absent tasks, 12 slots, DM + none used 0.78× the tokens of BM25 and DM 1.28×, as the agent stopped searching sooner. Both DM rankers used 0.78–0.81× on the 7 multi-service tasks, 0.90–0.92× on the 23 single-service tasks and 0.93–0.97× on the 4 controls. The Claude Code-style ranker made the fewest LLM calls, 0.82× [0.75, 0.90], and was fastest, 0.81× wall time [0.72, 0.90], because agents picked tools from the name list and searched 6.0 times per run against 11.4; but every prompt carried 1,076 names, a median of 25.8k tokens per call, so its tokens tied with BM25. The DM adds about 0.5 s per search; wall time with the DM rankers did not differ significantly from BM25: 1.04× [0.90, 1.19] for DM + none and 1.03× [0.94, 1.14] for DM.

Table 4: Search accuracy and effort in the main run: 4,785 searches from the 408 runs of four rankers. Hit: the five results contain a tool of the oracle label, over the 2,862 searches made while a needed tool was missing. Live: results agents received; replay: every query sent to every ranker; by writer: range over the rankers under which the queries were written. Searches per needed tool: searches made while working towards each of the 228 needed tools per ranker; mean, and share needing three or more. "None": DM + none's empty answers among its 1,147 live searches, the 2,862 replayed queries, and the 879 searches from tasks that need no tool. Bold: best.

	Hit			Searches per needed tool		
Ranker	live	replay	by writer	mean	≥3	"None": live / replay / no-tool
BM25	0.23	0.24	0.21–0.25	3.43	46%	–
DM	0.27	**0.29**	0.27–0.30	**2.73**	**30%**	–
DM + none	**0.28**	0.28	0.27–0.29	3.14	32%	5.6% / 4.1% / 9%
embeddings	0.25	0.24	0.21–0.25	3.09	41%	–

Table 5: *Projected* LLM cost per run with BM25 and with DM + none in the bridge, the DM fee of $0.0020 per run included, if each model used the mean token mix measured with DeepSeek V4.1 Flash in the main run. Prices: OpenRouter list prices on 5 November 2026 in $ per million tokens. Saving per 1,000 runs is BM25 minus DM; negative means the DM costs more. [a]Gemini 4 Argon has the same list prices as Claude Sonnet 5.5 and GPT-6.1 Sol, and hence the same projection.

Agent model	Price (in / cached / out)	BM25 $/run	DM $/run	Change	Saving/1k runs
DeepSeek V4.1 Flash (measured)	0.15 / 0.003 / 0.60	0.0144	0.0157	+9%	−$1.3
Claude Haiku 5.5	0.10 / 0.01 / 0.50	0.0168	0.0175	+4%	−$0.7
Claude Sonnet 5.5[a]	2.00 / 0.10 / 10.00	0.261	0.246	−6%	$15
Kimi K3.1 Pro	2.00 / 0.20 / 10.00	0.336	0.312	−7%	$24
GPT-6.1 Sol[a]	2.00 / 0.10 / 10.00	0.261	0.246	−6%	$15
Claude Fable 5.1	10.00 / 0.25 / 50.00	1.114	1.057	−5%	$58

Search accuracy in the live runs. We gave every search an oracle label: the needed-tool group the agent was working towards, from the task's list of needs and the agent's successful calls so far. Whichever ranker's agent wrote the queries, the DMs returned a tool of that group more often than BM25 and embeddings (Table 4); on replayed queries DM was right and BM25 wrong 228 times, and the reverse 84 times, consistent with the fewer searches per needed tool. Rates are low because many searches are exploratory, such as "list all available services" or guesses at services the company does not use: every ranker missed 1,890 queries. The live "none" answers of DM + none we sampled asked for services absent from the catalog, such as AWS ECS, Google Drive and a Datadog-like APM.

5.2 Projection: cost with other agent models

This is an analysis of the main run, not a new measurement: at which agent-model price is the DM's fee repaid by the tokens it saves? We take the mean token mix per run over all 114 runs of each ranker: BM25 used 34.6k uncached input, 754.5k cached input and 11.6k output tokens including reasoning, DM + none 33.3k, 659.7k and 11.1k. The DM's fee, as billed in the run, was $0.0002 per search, $0.0020 per run at 10.1 searches. We price both mixes at each model's OpenRouter list price on 5 November 2026 and add the fee to the DM column (Table 5). This assumes a frontier agent would produce a similar token mix and reduction; stronger models may instead search less or write better BM25 queries. Since the token reduction itself is not significant after correction for multiple comparisons, the projection is indicative.

Table 6: Offline retrieval on 120 hand-written queries. Hit@k: a right tool is among the top k. Cost per 1,000 queries and median latency of the selecting call at 613 tools; name, BM25 and embedding search run locally, and embedding cost is negligible. Bold: best value per accuracy column.

	613 tools			1,076 tools			
Mechanism	hit@1	hit@5	MRR	hit@1	hit@5	$/1k queries	latency (s)
Claude Code style	0.48	0.71	0.56	0.45	0.65	0	0.01
BM25	0.54	0.75	0.62	0.49	0.76	0	<0.01
embeddings	0.51	0.80	0.61	0.47	0.75	≈0	0.67
LLM router	0.86	**0.98**	0.92	0.82	**0.98**	0.44	1.31
DM hierarchy	**0.93**	0.97	**0.95**	**0.88**	0.97	0.15	0.54
DM tournament	0.90	**0.98**	0.94	0.86	**0.98**	1.13	1.60
BM25→DM	0.87	**0.98**	0.91	0.80	0.96	0.18	0.33
embed→DM	0.88	**0.98**	0.93	0.80	**0.98**	0.18	0.99

The fee is fixed; the saving scales with the agent's price. At these prices the DM + none mix costs 5–8% less than BM25's before the fee, less than the token reduction because cached input, which fell most, by 13%, is cheap and output fell only 4%. The fee equals this saving at an agent-LLM cost of $0.026–0.038 per run, depending on the model's price ratios, so break-even lies at about $0.03. Below it, as with the two inexpensive models, the DM makes a run 4–9% more expensive; with the mid-range and frontier models, 5–7% cheaper, saving $15–58 per thousand runs. The fee could be lowered by returning exact tool names found in the query without a DM call, caching repeated queries, or answering from the server step alone when the DM is confident.

5.3 Why the DM ranks well: offline retrieval

Offline retrieval removes the agent, so that ranking quality can be measured on fixed queries without agent noise: it tests which ranker puts the right tool on top, and which of the four strategies of §3 takes the DM past 255 options. We wrote two queries for each needed step of the v1 tasks, one in natural language and one as keywords, 120 in all, and counted a hit when the top k contained any acceptable tool for that step, at 613 and 1,076 tools; the LLM router uses DeepSeek V4.1 Flash (Table 6, App. E).

The DM ranks best; lexical and embedding searches rank worst. At 613 tools the Microsoft Decision 1 hierarchy put a right tool first for 93% of queries, the Claude Code-style, BM25 and embedding mechanisms 48–54%, the LLM router 86%; learned rankers reached hit@5 of 0.97–0.98, the others 0.71–0.80. The weak embedding results agree with ToolRet, where general-purpose retrievers do poorly on tool retrieval [27]. The DM hierarchy cost $0.15 per thousand queries, a third of the router's cost, at under half its latency. On 183 queries written by agents in our live runs the DM's lead is smaller (App. C).

Choosing the server first is the right way past 255 options. The hierarchy was the most accurate strategy at hit@1 and the cheapest. The tournament matched its hit@5 at seven times the cost, since it queries the DM for every chunk; BM25 or embedding shortlists were as cheap but lost 5–6 points at hit@1 when the shortlist dropped the right tool. Most needs also lie within one server.

Table 7: opencode on task set v1, 613 tools, two repeats per task. Pass: runs passed on the 25 tasks that need a catalog tool. Largest prompt: median over those runs. Cost: mean LLM cost per run over all 33 tasks, excluding the selector, which cost $0.0011 per run with DeepSeek and $0.0031 with hy4 for the LLM router and $0.0003 and $0.0010 for the DM. Bold: best pass rate and smallest largest prompt among conditions with catalog access; *none* has no catalog and passes no tool task.

	DeepSeek V4.1 Flash			Tencent hy4		
Condition	Pass	Largest prompt	$/run	Pass	Largest prompt	$/run
none	0/25	9.6k	0.0110	0/25	15.4k	0.0154
native	**25/25**	181.9k	0.0722	**25/25**	188.5k	0.1039
cc	**25/25**	16.2k	0.0089	**25/25**	**32.0k**	0.0375
bm25	**25/25**	13.4k	0.0093	**25/25**	33.6k	0.0417
groups	**25/25**	29.0k	0.0179	24/25	52.7k	0.0562
embed	**25/25**	13.3k	0.0090	23/25	49.0k	0.0514
llm	**25/25**	**11.7k**	0.0076	**25/25**	32.6k	0.0419
dm	**25/25**	14.0k	0.0082	**25/25**	34.6k	0.0470

Decoy servers cost every mechanism a little. Adding 463 tools, including GitLab, CloudWatch and EKS beside GitHub, Loki and Kubernetes, lowered hit@1 by 3–6 points for every mechanism and left the order unchanged.

5.4 Eight mechanisms in a live agent: opencode

This experiment checks the premises of the main one in a second harness: that the tasks need the catalog, and how much context search saves against native loading of every definition, which the main experiment does not include. Each of the 33 v1 tasks ran under the eight conditions of Table 1, including *none* without a catalog, with two LLMs, DeepSeek V4.1 Flash and Tencent hy4, a second inexpensive model used only here, twice per task and model: 528 runs over 613 tools (Table 7, App. E).

The tasks need the catalog, and every mechanism provides it. Without a catalog none of the 50 tool-task runs passed; with one, every condition passed 96–100% (Table 7). All conditions abstained on every run of the four absent tasks, and no control run made a catalog call. The three failures were hy4 runs: under groups and under embed the agent also restarted a production Kubernetes deployment unasked, a destructive call, and once under embed it stopped searching before finding the ticket's specification.

Search removes most of the context. Loading every definition made the largest prompt about 182–189k tokens; with ranked search it was 12–16k with DeepSeek and 32–35k with hy4, and 29–53k with groups, whose activation loads a whole server: 5–16× less. The cost per run fell from $0.072 to $0.008–0.009 for DeepSeek and from $0.104 to $0.038–0.047 for hy4. Differences among ranked searches are within run-to-run noise.

Runs use few tools. The median catalog run used 4 distinct catalog tools in about 10 calls, almost always from one server; 23 of the 25 v1 tool tasks need only one server. Choosing the server is most of the problem, which suits the DM hierarchy but is also easy for any mechanism with a service overview; hence task set v2 and the larger catalog in the main experiment.

6 Threats to validity and lessons

Ceiling, power and multiple comparisons. Pass rates were at ceiling on v1 and, despite the harder v2 tasks, in the main experiment, so we rank by tokens, which are noisy. With 114 pairs per comparison the main experiment is slightly short of the roughly 136 needed to detect a 10% token difference. Only random's difference from BM25 survives correction for the five comparisons; the 7–12% reductions of the learned rankers are a consistent lean, not an established effect.

Scheduling. BM25, DM and random were queued back to back within each slot; DM + none, Claude Code style and embeddings ran later in parallel batches on the same pinned endpoint. Time-varying provider load, caching and latency could therefore affect comparisons involving the later three, including DM + none against BM25.

Simulated answers. Real services return longer, messier results, which would raise token counts and could change how agents search; empty results for unmatched reads may make absent tasks easier to recognize.

The connected-services note. The note that starts every prompt of the main experiment (§4) stands in for what real harnesses say in the system prompt, since agents not told what exists may never search (App. F), but it may raise search rates above those of an unprompted agent.

Approximated and omitted mechanisms. We reimplemented Claude Code's ToolSearch, Copilot's virtual tools and Goose's routers from their documentation; only Hermes's bridge is shipped code, and our cc scoring in particular is an approximation. opencode conditions return full definitions and route calls through `call_tool`, Hermes returns compact hits and typed calls, so we compare rankers only within one harness. The LLM router and groups were tested offline and in opencode but not in the main experiment (§5.1).

Query distribution and scale. Hand-written queries overstate the DM's offline advantage (App. C). The live results rest on two inexpensive LLMs and one DM, the main experiment on one LLM.

7 Related work

Tool retrieval for large catalogs. Retrieving tools before generation is the standard answer to large catalogs [7, 25]. AnyTool [9] selected among over 16,000 APIs with a hierarchy of LLM agents, category first and tool second; MCP-Zero [10] brings the same server-then-tool idea to MCP, letting the agent request tools by describing what it needs. Our DM applies this hierarchy with a decision model instead of an LLM. RAG-MCP [11] retrieves MCP tool descriptions with embeddings to reduce prompt bloat, and Mudunuri et al. [19] select among 121 tools from five MCP servers with embeddings, reporting retrieval accuracy and token reduction but no agent task runs. ToolRet finds that embedding retrievers strong on general benchmarks do poorly on tool retrieval, with a best nDCG@10 of 33.8 [27], and Lee [13] find that selecting a tool is largely retrieval. Agent skills [1] apply progressive disclosure to instructions, keeping a one-line summary in context and loading the body on demand; our service overviews and the v2 handbook play this role for the catalog. We compare such ideas with shipped mechanisms inside unmodified agents.

MCP benchmarks. LiveMCPBench [17] evaluates agents navigating many real MCP servers, MCPMark [30] stress-tests realistic MCP use with verifiable tasks, and MCP-Atlas [5] and Toolathlon [14] score task completion with real tools. They measure end-to-end success; we fix the agent, vary only the selection mechanism, and simulate back ends so runs are reproducible and writes can be graded safely.

Vendor reports. Stacklok compared its MCP Optimizer with Anthropic's tool search tool on 2,792 tools and reported much higher selection accuracy for its hybrid retrieval [28]; Arcade tested Anthropic's tool search on about 4,000 tools [12]. Both measure selection accuracy, not task completion, and neither reports token costs per task.

Decision models. The Artificial Analysis Decision Index [4] and an analysis of it [21] compare DMs with LLMs on standalone decisions, including tool calling; we place a DM inside agents as the tool-search ranker.

8 Conclusion

A decision model can serve as a coding agent's tool selector. On real MCP tool definitions, Microsoft Decision 1 ranks tools more accurately than the lexical and embedding searches current harnesses use and more cheaply than an LLM router, and asking for the server first takes it past its 255-option limit. In live agents, however, every search mechanism reaches ceiling pass rates, and agents' own queries narrow the ranking advantage. Inside Hermes Agent's bridge the robust result is that ranking matters: random ranking cost 57% more tokens than BM25. The Microsoft Decision 1 rankers and embeddings leaned 7–12% below BM25, with fewer LLM calls, smaller prompts and, for the DM, fewer searches per needed tool; only the variant with a "none" option had a 95% CI excluding 1, at 0.88× [0.78, 0.98], and not after correction for multiple comparisons. If a reduction of this size holds for other agent models, the DM would make runs 5–7% cheaper for mid-range and frontier models and 4–9% more expensive for inexpensive ones such as the model we measured (§5.2). Confirming it needs more paired slots and conditions run in the same schedule.

Reproducibility. We will release the tool harvester, catalog snapshots, simulator, the 38 tasks in both prompt versions with hidden tests and reference solutions, the MCP server with all conditions, the Hermes patch, the sandbox setup, all run records, and the analysis and figure scripts.

References

[1] Anthropic. Agent skills overview (Claude platform documentation). https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview, 2025. Accessed 6 November 2026.

[2] Anthropic. Tool search tool (Claude platform documentation; regex and BM25 variants). https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-search-tool, 2025. Accessed 6 November 2026.

[3] Anthropic. Claude code documentation: Scale with MCP tool search. https://code.claude.com/docs/en/mcp, 2026. Accessed 6 November 2026.

[4] Artificial Analysis. Artificial analysis decision index. https://artificialanalysis.ai/evaluations/artificial-analysis-decision-index, 2026. Accessed 6 November 2026.

[5] Chaithanya Bandi et al. MCP-Atlas: A large-scale benchmark for tool-use competency with real MCP servers. *arXiv preprint arXiv:2602.00933*, 2026.

[6] Block. goose: Tool selection strategy (vector and LLM-based tool routers, preview feature). https://web.archive.org/web/20251031035432/https://block.github.io/goose/docs/guides/managing-tools/tool-router/, 2025. Preview in 2025. The vector strategy was removed in https://github.com/block/goose/pull/3933 (August 2025), the LLM-based strategy and its documentation in https://github.com/block/goose/pull/6250 (December 2025).

[7] Huamin Chen, Xunzhuo Liu, Junchen Jiang, Bowei He, and Xue Liu. Outcome-aware tool selection for semantic routers: Latency-constrained learning without LLM inference. *arXiv preprint arXiv:2603.13426*, 2026.

[8] DeepSeek. DeepSeek V4.1 Flash. https://openrouter.ai/deepseek/deepseek-v4.1-flash-1017, 2026. OpenRouter model page; model identifier `deepseek/deepseek-v4.1-flash-1017`.

[9] Yu Du, Fangyun Wei, and Hongyang Zhang. AnyTool: Self-reflective, hierarchical agents for large-scale API calls. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*, pages 11812–11829, 2024. arXiv:2402.04253.

[10] Xiang Fei, Xiawu Zheng, and Hao Feng. MCP-Zero: Active tool discovery for autonomous LLM agents. *arXiv preprint arXiv:2506.01056*, 2025.

[11] Tiantian Gan and Qiyao Sun. RAG-MCP: Mitigating prompt bloat in LLM tool selection via retrieval-augmented generation. *arXiv preprint arXiv:2505.03275*, 2025.

[12] Eric Gustin. We threw 4,000 tools at Anthropic's new tool search. Here's what happened. https://www.arcade.dev/blog/anthropic-tool-search-4000-tools-test/, December 2025. Arcade.dev blog, 3 December 2025.

[13] Janghoon Lee. Selection is retrieval, abstention is not: On-device tool routing over 70 Korean-English actions. *arXiv preprint arXiv:2609.18672*, 2026.

[14] Junlong Li et al. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.25726.

[15] Microsoft. Use tools with agents (Visual Studio Code documentation: 128-tool limit and virtual tools). https://code.visualstudio.com/docs/copilot/agents/agent-tools, 2026. Accessed 6 November 2026.

[16] Microsoft. Not every decision needs an LLM: Getting started with Microsoft-Decision-1. https://techcommunity.microsoft.com/blog/educatordeveloperblog/not-every-decision-needs-an-llm-getting-started-with-microsoft-decision-1/4563468, October 2026. Microsoft Community Hub, Educator Developer Blog.

[17] Guozhao Mo, Wenliang Zhong, Jiawei Chen, Qianhao Yuan, Xuanang Chen, Yaojie Lu, et al. LiveMCPBench: Can agents navigate an ocean of MCP tools? *arXiv preprint arXiv:2508.01780*, 2025.

[18] Model Context Protocol. Model context protocol specification. https://modelcontextprotocol.io/specification/2025-06-18, 2025.

[19] Sarat Mudunuri, Jian Wan, Ally Qin, and Srinivasan Manoharan. Semantic tool discovery for large language models: A vector-based approach to MCP tool selection. *arXiv preprint arXiv:2603.20313*, 2026.

[20] Nous Research. Hermes agent. https://github.com/NousResearch/hermes-agent, version 0.19.0 (PyPI), 2026.

[21] Randal S. Olson. When to use a decision model instead of an LLM. Blog post, 15 October 2026, https://www.randalolson.com/2026/10/14/when-to-use-a-decision-model-instead-of-an-llm/, 2026.

[22] OpenAI. Codex CLI. https://github.com/openai/codex, 2026. MCP tool search with BM25 in `codex-rs/core/src/tools/handlers/tool_search.rs`; source checked 6 November 2026.

[23] opencode contributors. opencode: an open-source coding agent. https://opencode.ai, version 1.19.4, 2026.

[24] OpenRouter. Decisions API reference. https://openrouter.ai/docs/api/api-reference/alphadecisions/submit-a-decisions-request, 2026. Accessed 6 November 2026.

[25] Yujia Qin et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2307.16789.

[26] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[27] Zhengliang Shi, Yuhan Wang, Lingyong Yan, Pengjie Ren, Shuaiqiang Wang, Dawei Yin, and Zhaochun Ren. Retrieval models aren't tool-savvy: Benchmarking tool retrieval for large language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 24497–24524, 2025. arXiv:2503.01763.

[28] Stacklok. Stacklok's MCP optimizer vs Anthropic's tool search tool: A head-to-head comparison. https://stacklok.com/blog/stackloks-mcp-optimizer-vs-anthropics-tool-search-tool-a-head-to-head-comparison/, 2025.

[29] TypeSafe AI. Jev 1.14 decision model. https://typesafe.ai; served via OpenRouter as `typesafe/jev-1.14`, https://openrouter.ai/typesafe/jev-1.14, 2026.

[30] Zijian Wu, Xiangyan Liu, Xinyuan Zhang, Lingjun Chen, Fanqing Meng, Lingxiao Du, et al. MCPMark: A benchmark for stress-testing realistic and comprehensive MCP use. *arXiv preprint arXiv:2509.24002*, 2025.

A Harness hardening

The setup of §4 is the result of fixes made during development; runs made before them are not reported. In order:

- Agents run as an unprivileged user with an empty environment and no credentials; experiment files are readable only by root and the API key is held by a local proxy that admits only the experiment models, because harness-level permissions alone let an agent read hidden tests through absolute paths.
- Each Hermes run gets its own mount namespace: a private work directory, home directory and `/tmp`, so runs cannot see each other's files or leftovers.
- The MCP server accepts only requests carrying a per-run token that the agent's shell cannot read, so the shell cannot list the catalog or query the ranker directly.
- Hermes's `approvals.deny` list blocks probing commands: `curl`, `wget`, `ps`, `ss`, `netstat`, `nmap`, `lsof`, `/proc`, `localhost` and `find /`.
- Rankers are compared inside Hermes's own bridge rather than through a separate MCP-side search with a generic `call_tool`, so that only the ranking differs.
- The provider and sampling are pinned as described in §5.1.

B Next-tool prediction in a real session

This experiment motivates using the DM as a selector rather than as the caller (§3). We took the 395 tool decisions of one real coding session and asked each model to predict the next tool among the 38 available. MSD1 reached accuracy 0.49 and TypeSafe Jev [29] 0.60, both below "always the shell" at 0.75, although the DMs' top five contained the right tool 94–96% of the time. In that session 51 always-loaded tool definitions took about 59k tokens, 93% of the first request and 16% of all context tokens read.

C Offline retrieval on agent-written queries

This experiment checks whether the offline ranking of §5.3 carries over to the queries agents actually write. We took 183 queries from the searches in our live runs, mostly keyword lists and tool-name fragments, labelled each with the needed tool the agent was working towards, and ran them through the same mechanisms. The DM still led at hit@1, 0.60 against 0.45 for BM25, but barely at hit@5, 0.70 against 0.66. Keyword-style queries suit lexical search, and agents compensate for a weak first query by searching again, so offline ranking quality translates only partly into agent behaviour.

D Hermes pilot

Before the main experiment we ran Hermes Agent with BM25 and DM over 613 tools on 12 v1 tasks with two repeats, 24 paired slots in all, analysed as in §5.1. It pointed the same way: DM used 0.88× the tokens of BM25, 95% CI [0.77, 1.02].

E Figures for the supporting experiments

Figures 3 and 4 plot Tables 6 and 7.

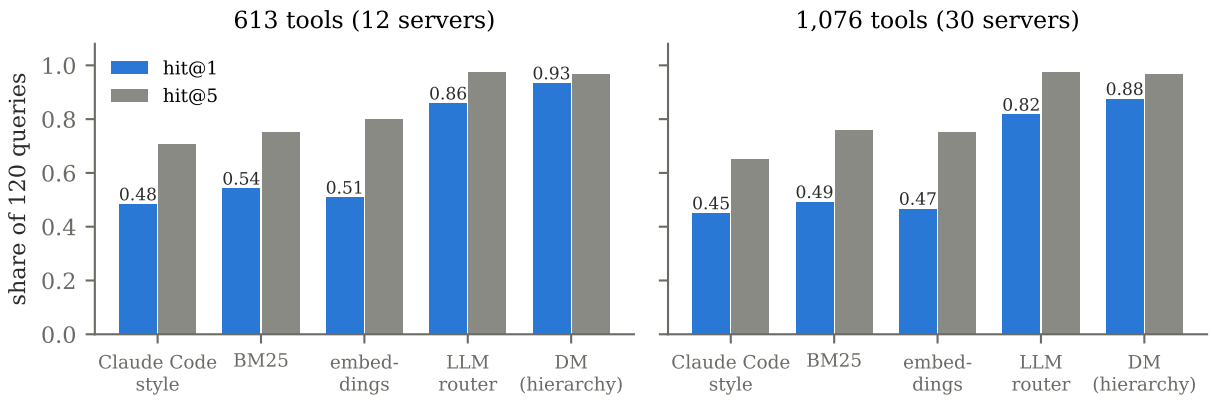


Figure 3: Offline retrieval on the main and the extended catalog: hit@1, labelled, and hit@5.

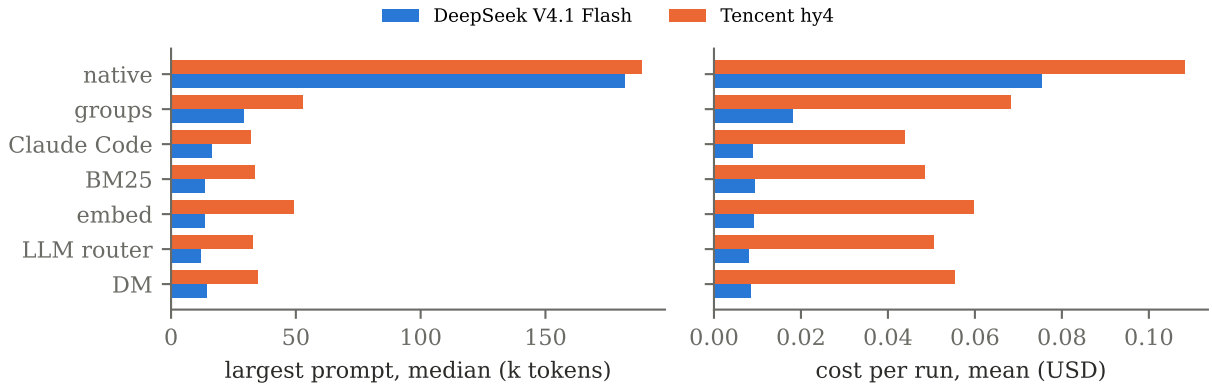


Figure 4: opencode, task set v1, 613 tools: median largest prompt on the 25 tool tasks and mean cost per run, by condition and LLM.

F The earlier synthetic pilot

An earlier version of this study used a fictional company's catalog of 100 and 300 synthetic tools of about 610 tokens each with opencode, and compared loading all tools, a DM-backed `find_tool`/`call_tool` pair with and without a service list, and tools preselected from the task prompt. It showed the same pattern as the real-catalog runs: search matched native loading at a fraction of the context, and an agent without a list of the services sometimes never searched. We replaced it because its templated descriptions were more uniform and easier to rank than real ones, its tasks were at ceiling, and its tools resembled no real server, so it could not compare the mechanisms that harnesses ship. Its code and results remain in the repository.

G Tool catalog and task set v2

Table 8: The real tool catalog: tools per MCP server, from each server's own `tools/list`.

Catalog	Servers (tools)	Tools
main	Linear (198), Atlassian (98), GitHub (91), Grafana (81), Git (28), MongoDB (27), Playwright (25), Notion (24), Kubernetes (23), Sentry (9), Slack (8), Postgres (1)	613
extra	GitLab (127), monday (84), Salesforce (74), Heroku (33), HubSpot (21), AWS CloudWatch (19), AWS EKS (16), Airtable (16), Filesystem (14), CircleCI (13), Browserbase (9), Memory (9), Netlify (9), Terraform (9), Elasticsearch (4), ClickHouse (3), Figma (2), Docker (1)	+463
all	30 servers	1,076

Table 9: Task set v2 (38 tasks). Prompts are shortened to their first sentence and shown without the escape clause. Services: the servers of the needed tools, in order; $+w$: number of required writes; $\star$: new in v2.

Category	Task	Prompt (first sentence)	Services
control	`c_chunk`	chunk in batching/chunk.py drops the last partial chunk.	–
control	`c_dedupe`	dedupe in collections_util/dedupe.py loses the original order.	–
control	`c_duration`	parse_duration in timeutil/duration.py only understands hours.	–
control	`c_leap`	is_leap in calendar_util/leap.py is wrong for some years.	–
single	`confluence_limits`	Our public API rate limits changed.	atlassian
single	`eks_importer` $\star$	The data team's nightly batch-importer job has failed three nights in a row.	aws_eks
single	`gh_ci_fail`	Pull request #214 in acme/payments can't be merged because a check is red.	github $+1w$
single	`gh_codesearch`	SUPPORTED_CURRENCIES in shop/currencies.py has drifted from the company-wide canonical list, which lives in another repository.	github
single	`gh_dependabot`	Security says acme/payments has dependencies with known vulnerabilities.	github
single	`gh_issue`	Please implement acme/payments#301.	github
single	`gh_open_pr`	Pagination in acme/catalog is broken (see #57).	github $+2w$
single	`gh_release`	We are behind on the ledger API.	github
single	`gh_review`	There is reviewer feedback waiting on pull request #88 in acme/ledger.	github $+1w$
single	`grafana_incident`	We are in the middle of an incident about checkout timeouts and the responders agreed on a mitigation.	grafana $+1w$
single	`jira_impl`	Pick up PAY-412: implement it, then move it to review.	atlassian $+1w$
single	`k8s_crashloop`	ledger-worker keeps restarting in production (namespace ledger) since the last config change.	kubernetes
single	`k8s_oom`	report-builder (namespace reports) keeps dying in production.	kubernetes

Category	Task	Prompt (first sentence)	Services
single	`linear_bug`	Fix ENG-77 and move it to In Review.	linear +1w
single	`loki_errors`	Customers stopped getting shipping e-mails at around 06:00 UTC today; the notifier service is the suspect.	grafana
single	`mongo_shape`	The nightly export has started crashing on recent orders.	mongodb
single	`notion_runbook`	We are rotating the webhook signing keys.	notion
single	`oncall_contact`	Set ESCALATION_CONTACT in ops/escalation.py to the e-mail address of whoever is on call for the payments team right now.	grafana
single	`pg_migration`	Add migrations/0042_payments_merchant_idx.sql with an index named payments_merchant_created_idx for looking up a merchant's payments by creation time.	postgres
single	`prom_threshold`	Set SLOW_REQUEST_MS in alerts/config.py to twice the checkout service's current p99 request latency, in milliseconds.	grafana
single	`sentry_events`	Since this morning's deploy some clients get HTTP 500 from POST /refunds.	sentry
single	`sentry_fix`	The noisiest production error of payments-api right now needs to go.	sentry +1w
single	`slack_decision`	The payments team agreed on a new webhook retry policy in a chat discussion.	slack +1w
multi	`cw_reports` ⋆	The legacy reports API has been answering 502 since yesterday evening.	aws_cloudwatch, slack +1w
multi	`gl_pipeline` ⋆	The main-branch pipeline of billing-legacy is red.	gitlab +1w
multi	`linear_notion_pr` ⋆	Do ENG-91 the way our API guidelines say, and open a pull request for it from a branch named eng-91 against main in acme/public-api.	linear, notion, github +1w
multi	`multi_alert`	Something is paging for the checkout service.	grafana, slack +1w
multi	`multi_jira_conf`	Implement PAY-530 as its design document specifies, then leave a one-line summary on the ticket.	atlassian +1w
multi	`multi_sentry_gh`	A recent change to ledger-worker introduced a reconciliation bug that shows up as a production error.	sentry, github +1w
multi	`promo_sentry_jira` ⋆	Android users report they can't apply promo codes.	sentry, atlassian +1w
absent	`absent_jira`	Implement PAY-999 in payments/limits.py.	–
absent	`absent_release`	Pin SDK_VERSION in sdk/version.py to the newest release of acme/ghost-sdk (without the leading v).	–
absent	`absent_sentry`	The billing-cron error someone mentioned in standup needs fixing in billing/cron.py.	–
absent	`absent_slack`	Set CACHE_TTL_S in cache/config.py to the value the platform team agreed on.	–